

Response Surface Designs: A review and an example for educational purposes

Kontadakis Eftychios

Department of Applied Mathematical and Physical Sciences, National and Technical University of Athens

felkon0@gmail.com

Abstract

This work reviews Response Surface Designs (RSDs), explains their theoretical foundations, and demonstrates their application through artificially generated data. A Central Composite Design is applied to recover the underlying quadratic model. The process is thoroughly documented to serve as educational material for anyone interested in this type of designs and how it is implemented.

Introduction

In this work we are going to review Response Surface Designs(RSDs), the theory behind them and some applications that distinguish them as a powerful tool both for research and industrial purposes. The current work's objective is purely educational and can be used as educational material freely.

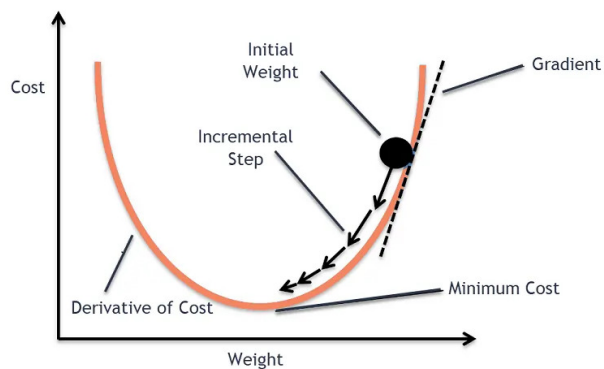


Figure 1: Gradient Descent Concept [1]

Purpose of RSDs

RSDs are generally constructed to efficiently estimate the parameters of a second-order model. They are widely used in research, industry, and engineering to identify the combination of factors that maximizes or minimizes a response. Because they include second-order (quadratic) terms, RSDs can detect curvature in the response surface. This allows sequential optimization strategies such as Steepest Ascent or Steepest Descent, commonly used in machine learning optimization, to be applied in order to locate critical points (typically maxima or minima) [3].

Types of RSDs

The most commonly used RSDs are:

- Central Composite Designs(CCDs)
- Box-Behnken Designs(BBDs)
- Doehlert Designs

each of these with its own advantages and disadvantages.

Central Composite Designs(CCDs)

This is the most common RSD, and the one we will mostly implement in this work. It is built specifically to estimate all terms of a full quadratic model:

$$y = \beta_0 + \sum_i \beta_i x_i + \sum_i \beta_{ii} x_i^2 + \sum_{i < j} \beta_{ij} x_i x_j + \varepsilon \quad (1)$$

There are a lot of different CCDs, so we will only mention the most commonly used ones.

Circumcribed Central Composite Designs(CCCs)

Specifically we are going to apply a special case of Central Composite Designs, namely Circumcribed Central Composite Designs(CCCs). In these designs, we may add points to explore beyond the natural limits, or in design terms, we add points outside the factorial cube expanding it[4]. These points are called axial points and are usually of the form:

$$A = \begin{bmatrix} +\alpha & 0 & 0 & \cdots & 0 \\ -\alpha & 0 & 0 & \cdots & 0 \\ 0 & +\alpha & 0 & \cdots & 0 \\ 0 & -\alpha & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & +\alpha \\ 0 & 0 & 0 & \cdots & -\alpha \end{bmatrix} \quad (2)$$

it can be proven that $\alpha = (n_f)^{1/4}$ where n_f is the number of factorial points.[4][3]

For this work a 2^2 factorial design is used as a the most simple example, thus these are all simplified to:

$$A = \begin{bmatrix} +\alpha & 0 \\ -\alpha & 0 \\ 0 & +\alpha \\ 0 & -\alpha \end{bmatrix} \quad (3)$$

where $\alpha = \sqrt{2}$. [4]

There are additional types of CCCs, namely Inscribed Central Composite Designs(CCI) and Face Centered Central Composite Designs (CCFs) each of them differentiating on where the axial points are placed.[4]

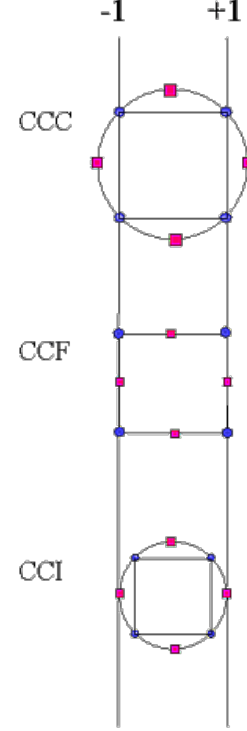


Figure 2: Visualization of Axial Points relative to the factorial cube.[4]

It is also worth noting that CCF is a scaled down version of CCC.

Box-Behnken Designs(BBDs)

Another class of response surface designs are Box-Behnken designs(BBDs). They are very useful in the same setting as the central composite designs. Their primary advantage is in addressing the issue of where the experimental boundaries should be, and in particular to avoid treatment combinations that are extreme. By extreme, we are thinking of the corner points and the axial points, which are extreme points in terms of region in which we are doing our experiment. The Box-Behnken design avoids all the corner points,

and the axial points.[6]. These designs are rotatable (or near rotatable) and require 3 levels of each factor. The designs have limited capability for orthogonal blocking compared to the central composite designs.[5]

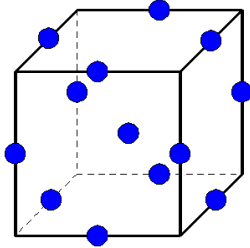


Figure 3: A Box-Behnken Design for Three Factors.[5]

The Doehlert design

The Doehlert design describes a spherical experimental domain and it stresses uniformity in space filling. Although this matrix is neither orthogonal nor rotatable, it does not significantly diverge from the required quality for effective use. For two variables, the Doehlert design consists of one central point and six points forming a regular hexagon, and therefore situated on a circle. In Doehlert designs the number of levels is not the same for all variables. That means that one variable can be studied at five levels while the other only three levels. This property allows a free choice of the factors to be assigned to a large or a small number of levels. Different criteria can be used to assign the factors. As a general rule, it is preferable to choose the variable with the stronger effect as the factor with five levels in order to obtain most information of the system. This characteristic makes this design unique and ideal for non symmetric designs.[2]

The Data

The data will be generated artificially, a model will be chosen arbitrarily and data will be generated from that model. The process will be

presented in detail to assist in replication. The model of choice is:

$$y = 80 + x_1 + x_2 - 0.05x_1^2 - 0.05x_2^2 + \varepsilon \quad (4)$$

Here x_1, x_2 take values $+1, -1, 0$, depending on whether they belong to a factor level or the center points. The most important part of this quadratic model is not the individual choices of parameters but the artificial noise inserted with the parameter ε . This parameter represents the residuals, the random error we would get if we were actually measuring the response in a lab. We create the design this way deciding the levels to be represented by $x_1, x_2 = +1, -1$ and the center points by $x_1, x_2 = 0$. Of course this choice is again arbitrary and we could have chosen other values, but we could easily convert any choice of factor levels ξ_1, ξ_2 back to $0, -1, +1$, with the transformation:

$$x_1 = \frac{\xi_1 + \frac{\xi_{1h} + \xi_{1l}}{2}}{\frac{\xi_{1h} - \xi_{1l}}{2}} \quad \& \quad x_2 = \frac{\xi_2 + \frac{\xi_{2h} + \xi_{2l}}{2}}{\frac{\xi_{2h} - \xi_{2l}}{2}} \quad (5)$$

where $\xi_{1h}, \xi_{1l}, \xi_{2h}, \xi_{2l}$ are the high and low level of each factor. We are now ready to generate the data for a 2^2 factorial design with 5 central points((0, 0)).

```
set.seed(2025) #Random seed inserted so
#that we get a controlled randomness
#that is replicable. This is done to
#check for errors in our code and to
#have the same data every time we run
#the R algorithm

# Factorial + center points
x1 <- c(-1, 1, -1, 1, rep(0, 5))
x2 <- c(-1, -1, 1, 1, rep(0, 5))

# True model without residuals. That's
#what we will try to regain in the end
#of this work.
y_true <- 80 + x1 + x2 - 0.05*x1^2 -
0.05*x2^2
```

```
# Random residuals we insert to simulate
#an actual experiment
eps <-
  rnorm(length(x1), mean = 0, sd = 0.25)

# Observed response(True plus random
#errors)
y <- y_true + eps

data <-
  data.frame(run = 1:9, x1, x2, y)
data
```

	run	x1	x2	y
1	1	-1	-1	78.05519
2	2	1	-1	79.90891
3	3	-1	1	80.09329
4	4	1	1	82.21812
5	5	0	0	80.09274
6	6	0	0	79.95929
7	7	0	0	80.09928
8	8	0	0	79.98000
9	9	0	0	79.91376

Curvature Testing

Now that we have the data, we forget the initial model and start a process of determining whether this data is best fitted with a linear model or a higher order one. To do that we will perform a linear regression using the central points to calculate how important a potential curvature term C is:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_C C + \varepsilon \quad (6)$$

```
# 1. Add curvature indicator to the
#central points cause only them can
#indicate curvature.
data$C <-
  ifelse(data$x1 ==+
    0 & data$x2 == 0, 1, 0)

# 2. Linear model with curvature term
curv_mod <- lm(y ~ x1 + x2 +
```

```
C, data = data)
```

```
out <- capture.output(summary(curv_mod))
cat("\\begin{group}\\scriptsize\\n")
```

```
cat(paste(out, collapse = "\\n"))
```

Call: lm(formula = y ~ x1 + x2 + C, data = data)

Residuals: 1 2 3 4 5 6 7 8 0.06778 -0.06778 -0.06778 0.06778
0.08373 -0.04973 0.09026 -0.02901 9 -0.09526

Coefficients: Estimate Std. Error t value Pr(>|t|)
(Intercept) 80.06888 0.04792 1670.972 1.46e-15 **x1 0.99464**
0.04792 20.757 4.81e-06 x2 1.08683 0.04792 22.681 3.10e-06
*** C -0.05986 0.06429 -0.931 0.395
— Signif. codes: 0 ‘’ 0.001 ‘’ 0.01 ‘’ 0.05 ‘’ 0.1 ‘’ 1

Residual standard error: 0.09584 on 5 degrees of freedom Multiple R-squared: 0.9947, Adjusted R-squared: 0.9916 F-statistic: 315.4 on 3 and 5 DF, p-value: 4.074e-06

```
cat("\\n\\end{group}")
```

It is visible that the curvature is not significant in this area of the initial model. This is due to the model's behavior mimicking linearity away from the crucial point. To visualize that we are going to plot the original model we generated artificially.

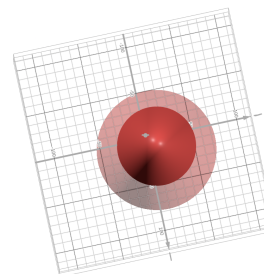
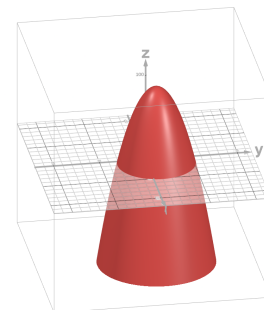


Figure 4: A 3D visualisation of the model, note the linearity of the model close to 0.

We will now move closer to the crucial point we should see the curvature become significant. Repeating the process but for a new set of factor levels, $\xi_{1h} = 8, \xi_{1l} = 4$ and $\xi_{2h} = 8, \xi_{2l} = 4$ thus the center point is now (6, 6). As discussed we can use the transformation 5 and return to $-1, +1, 0$. This time however it is not required since we only need to generate the new data in the same manner we did and check for the significance of curvature. The transformation will be done but no explicit calculations will be shown or are required since we are not going to keep the extracted linear model, we are only checking for curvature.

```
set.seed(2025)

# factorial + center around
#stationary point
x1 <- c(4, 8, 4, 8,
        rep(6, 5))
x2 <- c(4, 4, 8, 8,
        rep(6, 5))

y_true <- 80 + x1 + x2 -
  0.05*x1^2 - 0.05*x2^2
eps <- rnorm(length(x1), 0, 0.0025)
y <- y_true + eps
data_new <- data.frame(x1, x2, y)
```

With the data created, we once again try to fit a linear model with a curvature parameter.

```
# code around the *true* center
data_new$x1_c <-
  (data_new$x1 - 6) / 2
data_new$x2_c <-
  (data_new$x2 - 6) / 2
data_new$C <-
  ifelse(data_new$x1_c == 0 &
        data_new$x2_c == 0, 1, 0)

curv_mod_new <- lm(y ~ x1_c +
  x2_c + C, data = data_new)

# 3. Capture and print summary in a
out <-
```

```
capture.output(summary(curv_mod_new))
cat("\\begin{group}\\scriptsize\\n")
```

```
cat(paste(out, collapse = "\\n"))
```

Call: lm(formula = $y \sim x1_c + x2_c + C$, data = data_new)

Residuals: 1 2 3 4 5 6 7 0.0006778 -0.0006778 -0.0006778
0.0006778 0.0008373 -0.0004973 0.0009026 8 9 -0.0002901 -
0.0009526

Coefficients: Estimate Std. Error t value Pr(>|t|)
(Intercept) 8.800e+01 4.792e-04 183652.3 < 2e-16 $x1_c$
7.999e-01 **4.792e-04** **1669.4** **1.46e-15** $x2_c$ 8.009e-01 4.792e-
04 1671.3 1.46e-15 **C** **3.984e-01** **6.429e-04** **619.7** **2.08e-13**
— Signif. codes: 0 ‘**0.001**’ ‘**0.01**’ ‘**0.05**’ ‘**0.1**’ ‘**1**’

Residual standard error: 0.0009584 on 5 degrees of freedom
Multiple R-squared: 1, Adjusted R-squared: 1 F-statistic:
1.988e+06 on 3 and 5 DF, p-value: 1.311e-15

```
cat("\\n\\end{group}")
```

This time curvature is very significant and cannot be ignored, this means that we are going to need a higher order model, namely a second order one, but first we need to find the stationary point or at least the region of it to set the data cube there. Let's find the region of that stationary point by doing experiments while steadily increasing the factor inputs.

Search for the maxima

We will now take the extracted model from the first iteration where $x_1, x_2 = 0, +1, -1$ and slowly increase the parameters relative to each other, we will take all parameters as important apart from the curvature. Thus, the model is roughly:

$$y = 80 + x_1 + x_2 \quad (7)$$

Of course curvature here isn't included since it wasn't flagged as significant by the regression, we need to increase the two factors relative to each other, so if we increase x_1 by 1, that means that x_2 must be increased by their relative incline thus $1/1 = 1$. This time the coefficients are such that they should be increased together.

```

set.seed(2025) #the same seed

# Values from 1 to 12
# to check where the maximum is
vals <- 1:12

# the model
y_true <- 80 + vals + vals -
  0.05*vals^2 - 0.05*vals^2

# Random errors once again
eps <-
rnorm(length(vals), mean = 0, sd = 0.25)

# Observed responses
y_obs <- y_true + eps

# Build matrix for visualization
result_matrix <- cbind(
  x1 = vals,
  x2 = vals,
  y_obs = y_obs
)

result_matrix

```

	x1	x2	y_obs
[1,]	1	1	82.05519
[2,]	2	2	83.60891
[3,]	3	3	85.29329
[4,]	4	4	86.71812
[5,]	5	5	87.59274
[6,]	6	6	88.35929
[7,]	7	7	89.19928
[8,]	8	8	89.58000
[9,]	9	9	89.81376
[10,]	10	10	90.17554
[11,]	11	11	89.80108
[12,]	12	12	89.16124

We can see clearly the response rising up to the point (10, 10) after which it starts falling again. Thus the region we are going to work with must be around (10, 10) thus the levels are chosen to be (9, 11) for both factors. We can check for curvature again but we are already convinced that the model must be of higher order.

Applying the RSD

Thus we are going to apply an RSD, namely the CCC. So we will create another full factorial 2^2 with 5 center points and 4 axial points exactly as described in the theory. But first, we have to apply the transformation of 5 for our new factor levels to return to (0, -1, +1). This is now an important transformation as we will need to apply it backwards in our final model to extract its correct parameters (i.e. the ones we created ourselves).

Because the axial points require to take values ($\sqrt{2}$) after the transformation the data must be extracted at exactly the points: (11.414, 10), (8.586, 10), (10, 11.414), (10, 8.586) which after the transformation will become ($\sqrt{2}$, 0), ($-\sqrt{2}$, 0), (0, $\sqrt{2}$), (0, $-\sqrt{2}$). This is due to the transformation:

$$x_{1c} = \frac{x_1 - 10}{1}, \quad x_{2c} = \frac{x_2 - 10}{1}. \quad (8)$$

So once again we go to our “lab” and run the experiment once again but in the new factor levels, and we get the following data. It is important to note that in this section we reduced the errors by hand as they were overshadowing the curvature (in other words they were very big in relation to curvature itself).

```

set.seed(2025)

# Factorial points around (10,10)
x1 <- c(9, 11, 9, 11)
x2 <- c(9, 9, 11, 11)

# Add 5 center points
# exactly on (10,10)
x1 <- c(x1, rep(10, 5))
x2 <- c(x2, rep(10, 5))

# Add axial points (coded ±sqrt(2))
alpha <- sqrt(2)

x1 <- c(x1, 10+alpha, 10-alpha, 10, 10)
x2 <- c(x2, 10, 10, 10+alpha, 10-alpha)

```

```
#the model once again
y_true <- 80 + x1 + x2 -
  0.05*x1^2 - 0.05*x2^2

#the errors as usual
eps <-
rnorm(length(x1), mean = 0, sd = 0.005)
# we reduce noise substantially in the
#new area this is done by hand to
#enhance the model. Let's say we
#got better measuring instruments

#observed responses
y_obs <- y_true + eps

#The data
result_matrix <- cbind(
  x1 = x1,
  x2 = x2,
  y_obs = y_obs
)

result_matrix
```

	x1	x2	y_obs
[1,]	9.000000	9.000000	89.90310
[2,]	11.000000	9.000000	89.90018
[3,]	9.000000	11.000000	89.90387
[4,]	11.000000	11.000000	89.90636
[5,]	10.000000	10.000000	90.00185
[6,]	10.000000	10.000000	89.99919
[7,]	10.000000	10.000000	90.00199
[8,]	10.000000	10.000000	89.99960
[9,]	10.000000	10.000000	89.99828
[10,]	11.414214	10.000000	89.90351
[11,]	8.585786	10.000000	89.89802
[12,]	10.000000	11.414214	89.89122
[13,]	10.000000	8.585786	89.89790

Again we forget the model and try to guess it by applying the second order RSD:

```
# Fit a full quadratic
# response surface model
quad_mod <- lm(
  y_obs ~ x1 + x2 +
```

```
  I(x1^2) + I(x2^2)+x1*x2,
  data = as.data.frame(result_matrix)
)
```

```
out <-
capture.output(summary(quad_mod))
cat("\\begin{group}\\scriptsize\\n")
```

```
cat(paste(out, collapse = "\\n"))
```

Call: lm(formula = y_obs ~ x1 + x2 + I(x1^2) + I(x2^2) + x1 * x2, data = as.data.frame(result_matrix))

Residuals: Min 1Q Median 3Q Max -0.0057528 -0.0014091 -0.0002142 0.0018053 0.0059287

Coefficients: Estimate Std. Error t value Pr(>|t|)
(Intercept) 80.163680 0.311268 257.54 3.51e-15 **x1 0.952930**
0.037361 25.51 3.64e-08 x2 1.013764 0.037361 27.13 2.37e-08
I(x1^2) -0.048278 0.001558 -30.98 9.43e-09 I(x2^2)
-0.051382 0.001558 -32.97 6.11e-09 *** x1:x2 0.001356 0.002055
0.66 0.531

— Signif. codes: 0 ‘‘ 0.001 ’’ 0.01 ’’ 0.05 ’’ 0.1 ’’ 1

Residual standard error: 0.00411 on 7 degrees of freedom Multiple R-squared: 0.9962, Adjusted R-squared: 0.9934 F-statistic: 362.4 on 5 and 7 DF, p-value: 2.731e-08

```
cat("\\n\\end{group}")
```

It is clear that every term is significant apart from the interaction x_1, x_2 . We can also regain the initial model by applying the reverse transformation:

```
design <- as.data.frame(result_matrix)
```

```
# Coded variables around center (10,10)
```

```
design$x1_c <- design$x1 - 10
```

```
design$x2_c <- design$x2 - 10
```

```
quad_mod_coded <- lm(
  y_obs ~ x1_c + x2_c
  + I(x1_c^2) + I(x2_c^2)+x1_c*x2_c,
  data = design
)
```

```
out <-
capture.output(summary(quad_mod_coded))
cat("\\begin{group}\\scriptsize\\n")
```

```
cat(paste(out, collapse = "\\n"))
```

```
Call: lm(formula = y_obs ~ x1_c + x2_c + I(x1_c^2) +
I(x2_c^2) + x1_c * x2_c, data = design)
```

```
Residuals: Min 1Q Median 3Q Max -0.0057528 -0.0014091
-0.0002142 0.0018053 0.0059287
```

```
Coefficients: Estimate Std. Error t value Pr(>|t|)
(Intercept) 90.0001803 0.0018381 48963.019 < 2e-16 x1_c
0.0009168 0.0014532 0.631 0.548
x2_c -0.0003109 0.0014532 -0.214 0.837
I(x1_c^2) -0.0482784 0.0015583 -30.981 9.43e-09
I(x2_c^2) -0.0513815 0.0015583 -32.972 6.11e-09 *** x1_c:x2_c
0.0013556 0.0020551 0.660 0.531
— Signif. codes: 0 '0.001' '0.01' '0.05' '0.1' '1'
```

```
Residual standard error: 0.00411 on 7 degrees of freedom Multi-
ple R-squared: 0.9962, Adjusted R-squared: 0.9934 F-statistic:
362.4 on 5 and 7 DF, p-value: 2.731e-08
```

```
cat("\n\\endgroup")
```

Again, nothing changes in terms of significance it's only the coefficients that change due to the transformation. We are going to show them explicitly and cleanly.

```
# Extract coefficients from coded model
b <- coef(quad_mod_coded)

# Rename for clarity
b0 <- b["(Intercept)"]
b1 <- b["x1_c"]
b2 <- b["x2_c"]
b11 <- b["I(x1_c^2)"]
b22 <- b["I(x2_c^2)"]
b12 <- b["x1_c:x2_c"]

# Center point in natural units
c1 <- 10
c2 <- 10

# Build natural-unit coefficients
B0 <- b0 -
      b1*c1 -
      b2*c2 +
      b11*c1^2 +
      b22*c2^2 +
      b12*c1*c2

B1 <- b1 -
      2*b11*c1 -
      b12*c2
```

```
B2 <- b2 -
      2*b22*c2 -
      b12*c1
```

```
B11 <- b11
B22 <- b22
B12 <- b12
```

```
# Display the natural-unit model
cat("y = ",
    B0, " + ",
    B1, "*x1 + ",
    B2, "*x2 + ",
    B11, "*x1^2 + ",
    B22, "*x2^2 + ",
    B12, "*x1*x2\n")
```

$$y = 80.16368 + 0.95293 x_1 + 1.013764 x_2 + -0.04827844 x_1^2 + -0.05138154 x_2^2 + 0.001355562 x_1 x_2$$

It becomes evident that the extracted model is a good fit to the generated data as it strongly resembles the one we created.

$$y = 80 + x_1 + x_2 - 0.05 x_1^2 - 0.05 x_2^2.$$

Conclusion

In this work we generated a quadratic model of our choice, such that it has a maxima at some point. We tried to fit it with a linear model by doing a normal regression with central points to also calculate curvature relevance. Calculating curvature in an area away from the maxima, displayed no significance as expected, but moving closer to the maxima where we no longer have that linearity, the curvature term became important. We deduce that the data can't be fitted with a first order model, and we should switch to a second order one. We find the maxima with an algorithm similar to a steepest ascend method and move our data cube there. We apply the transformation and a CCC to fit the data. The extracted model strongly resembles the one originally created for

our artificial data, and it even showed the non-existing interaction term as insignificant making our overall fit almost perfect.

References

- [1] Crypto. *Gradient Descent Algorithm: How Does it Work in Machine Learning?* Apr. 2025. URL: <https://www.analyticsvidhya.com/blog/2020/10/how-does-the-gradient-descent-algorithm-work-in-machine-learning/>.
- [2] Sérgio L.C Ferreira et al. “Doehlert matrix: a chemometric tool for analytical chemistry—review”. In: *Talanta* 63.4 (Mar. 2004), pp. 1061–1067. DOI: [10.1016/j.talanta.2004.01.015](https://doi.org/10.1016/j.talanta.2004.01.015). URL: <https://doi.org/10.1016/j.talanta.2004.01.015>.
- [3] Douglas C. Montgomery. *Design and Analysis of Experiments*. Wiley, 2017.
- [4] NIST/SEMATECH. *Central Composite Designs*. <https://www.itl.nist.gov/div898/handbook/pri/section3/pri3361.htm>. Accessed: 2025-12-29. 2023.
- [5] NIST/SEMATECH. *Central Composite Designs*. Accessed: 29 December 2025. Engineering Statistics Handbook. 2025. URL: <https://www.itl.nist.gov/div898/handbook/pri/section3/pri3362.htm>.
- [6] Pennsylvania State University. *Box–Behnken Designs*. Accessed: 29 December 2025. STAT 503: Design of Experiments. 2025. URL: <https://online.stat.psu.edu/stat503/book/export/html/683>.